

Universidad de Costa Rica
Facultad de Ingeniería
Escuela de Ingeniería Eléctrica
IE 0435 – Inteligencia Artificial Aplicada a la Ingeniería Eléctrica
I ciclo 2026

Proyecto #2

Danna Guevara Quesada C23562
Grupo 01

Profesor: Marvin Coto Jiménez

Fecha de entrega: 01 de julio

Índice

1. Contextualización del Proyecto	1
1.1. Descripción general	1
1.2. Fuente de datos	1
1.3. Objetivo del agente	1
2. Preprocesamiento de Datos	2
2.1. Lectura y combinación de archivos	2
2.2. Extracción de variables temporales	2
2.3. Detección y corrección de anomalías	2
2.4. Cobertura temporal y justificación de los huecos en las gráficas 1 y 2.	3
2.5. Resultado del preprocesamiento	3
3. Entrenamiento y Selección de Modelos	4
3.1. Variables utilizadas	4
3.2. División de datos	4
3.3. Modelos evaluados	4
3.4. Resultados obtenidos	5
3.5. Predicciones contra demanda real	5
3.6. Comparación de métricas	6
4. Diseño del Agente	7
4.1. Arquitectura	7
4.2. Herramientas integradas	7
4.3. Ejemplo de interacción	7
5. Reproducibilidad del Proyecto	8
5.1. Repositorio	8
6. Limitaciones y Dificultades	8
7. Oportunidades de Desarrollo Futuro	9
8. Conclusiones	9

Índice de figuras

1. Curva de demanda real entre el 18 y el 22 de junio de 2026. Los cortes representan huecos temporales reales detectados en los datos.	5
2. Demanda real vs. predicciones de KNN y SVR sobre el conjunto de prueba.	5
3. Comparación de RMSE y MAE entre KNN y SVR.	6

1. Contextualización del Proyecto

1.1. Descripción general

El presente proyecto consiste en el desarrollo de un agente inteligente capaz de predecir la demanda eléctrica nacional de Costa Rica a corto plazo, específicamente para intervalos de 15 minutos, utilizando datos públicos del Centro Nacional de Control de Energía (CENCE). Este problema se enmarca dentro de las aplicaciones de series de tiempo en sistemas eléctricos de potencia, donde la predicción de la demanda permite apoyar la operación del sistema, anticipar variaciones de carga y mantener condiciones adecuadas de estabilidad.

Como guía se toma un proyecto eléctrico previo, el cual desarrolló una primera versión de un agente relacionado con predicción de demanda. A partir de esa base, se construyó una implementación propia con herramientas de aprendizaje automático supervisado y un agente capaz de seleccionar la herramienta de predicción más conveniente según el desempeño obtenido con los datos disponibles.

1.2. Fuente de datos

Los datos utilizados provienen directamente de la plataforma pública del CENCE:

- **Sitio web:** <https://apps.grupoice.com/CenceWeb/paginas/CurvaDemanda.html>
- **Formato de descarga:** archivo .txt delimitado por comas.
- **Variables incluidas:** demanda en MW, regulación arriba/abajo, fecha, ACE, frecuencia, intercambio con Panamá, intercambio con Nicaragua y reserva primaria.
- **Resolución temporal:** intervalos de 15 minutos.
- **Periodo utilizado para esta entrega:** combinación de 5 archivos descargados del CENCE, cubriendo datos desde el 18 de junio de 2026 a las 00:00 hasta el 22 de junio de 2026 a las 19:45.

1.3. Objetivo del agente

El objetivo del agente es seleccionar la mejor herramienta disponible para predecir la demanda eléctrica del siguiente intervalo de 15 minutos. Busca comparar el desempeño de distintos modelos de regresión, interpreta las métricas obtenidas y comunica el resultado de forma clara para un posible usuario del sistema eléctrico.

2. Preprocesamiento de Datos

Los archivos descargados del CENCE fueron procesados mediante el script `Preprocesamiento.py`. Este archivo localiza automáticamente todos los archivos que siguen el patrón `OperacionTiempoReal*`, valida sus columnas, convierte la fecha a formato temporal, convierte las columnas numéricas y combina todos los registros en un único conjunto ordenado cronológicamente.

2.1. Lectura y combinación de archivos

Para esta corrida final se combinaron 5 archivos:

- `OperacionTiempoReal`: 91 filas.
- `OperacionTiempoReal2`: 88 filas.
- `OperacionTiempoReal3`: 92 filas.
- `OperacionTiempoReal4`: 85 filas.
- `OperacionTiempoReal5`: 80 filas.

En total se combinaron 436 registros. No se encontraron fechas duplicadas entre archivos, por lo que no fue necesario eliminar registros repetidos. El rango temporal final del conjunto de datos fue desde el 18 de junio de 2026 a las 00:00 hasta el 22 de junio de 2026 a las 19:45.

2.2. Extracción de variables temporales

Dado que la demanda eléctrica presenta patrones diarios, se generaron variables temporales adicionales:

- `hora_decimal`: hora del día en formato decimal.
- `dia_semana`: día de la semana, donde 0 corresponde a lunes y 6 a domingo.
- `es_fin_semana`: variable binaria que indica si el registro corresponde a sábado o domingo.
- `gap_minutos`: diferencia en minutos respecto al registro anterior, utilizada para detectar huecos temporales.

2.3. Detección y corrección de anomalías

Se aplicó el método del rango intercuartílico, IQR, sobre la variable `demanda`. El procedimiento calcula el primer cuartil, el tercer cuartil y define límites inferior y superior mediante:

$$L_{inf} = Q1 - 1,5 \times IQR$$

$$L_{sup} = Q3 + 1,5 \times IQR$$

En esta corrida se obtuvieron los siguientes valores:

- $Q1 = 1290,71$ MW.
- $Q3 = 1807,63$ MW.

- $IQR = 516,92$ MW.
- Límite inferior: 515.34 MW.
- Límite superior: 2583.00 MW.

No se detectaron valores de demanda fuera de estos límites, por lo que no se corrigieron anomalías en la corrida final. Aun así, el código conserva la capacidad de corregir valores atípicos mediante recorte, evitando eliminar filas completas cuando solo una variable presenta un valor extremo.

2.4. Cobertura temporal y justificación de los huecos en las gráficas 1 y 2.

Aunque el conjunto final contiene datos desde el 18 hasta el 22 de junio de 2026, no todos los intervalos de 15 minutos están presentes. En una serie perfectamente continua entre la primera y la última fecha disponible se esperarían 464 registros, pero se obtuvieron 436 registros reales. Esto implica que existen 28 intervalos faltantes.

Estos intervalos faltantes justifican los huecos visibles en la gráfica de demanda histórica. Los cortes no representan un error de graficación ni una falla del modelo; representan momentos para los cuales no se dispone de medición en los archivos descargados. Por esta razón, la herramienta de visualización interrumpe la línea cuando detecta un salto temporal mayor a 15 minutos. Esta decisión es importante porque evita unir artificialmente dos puntos que no son consecutivos. Si la gráfica conectara esos puntos con una línea recta, se estaría sugiriendo una transición continua que realmente no fue observada.

El mismo criterio se utiliza durante el entrenamiento: las variables de rezago `lag_1`, `lag_2` y `lag_4` solo se calculan cuando los registros anteriores existen realmente a 15, 30 y 60 minutos, respectivamente. Si un rezago fuera a cruzar un hueco temporal, se descarta para evitar que el modelo aprenda relaciones falsas entre registros no consecutivos.

2.5. Resultado del preprocesamiento

Característica	Valor
Archivos combinados	5
Filas leídas por archivo	91 / 88 / 92 / 85 / 80
Registros totales combinados	436
Fechas duplicadas eliminadas	0
Anomalías corregidas	0
Valores nulos eliminados en columnas críticas	0
Registros esperados en serie continua	464
Intervalos faltantes	28
Rango de fechas	18–22 de junio de 2026
Demanda mínima / máxima	1067.01 / 1941.47 MW
Archivo de salida	<code>datos_limpios.csv</code>

Tabla 1: Resumen del preprocesamiento de datos en la corrida final.

3. Entrenamiento y Selección de Modelos

3.1. Variables utilizadas

El objetivo de los modelos es predecir la demanda del siguiente intervalo de 15 minutos. Las variables utilizadas fueron:

- `hora_decimal`, `dia_semana` y `es_fin_semana`, capturan patrones temporales.
- `frecuencia`, `intercambioSur`, `intercambioNorte` y `primaria`, representan variables del sistema eléctrico.
- `lag_1`, `lag_2` y `lag_4`, correspondientes a la demanda registrada 15, 30 y 60 minutos antes.

De los 436 registros limpios disponibles, 411 resultaron útiles para entrenamiento y prueba después de crear los rezagos y la variable objetivo. La diferencia se debe principalmente a los registros iniciales necesarios para formar rezagos y a la eliminación de casos donde los rezagos crucen huecos temporales.

3.2. División de datos

La partición entre entrenamiento y prueba se realizó de forma secuencial, sin mezclar aleatoriamente los datos. Así las series de tiempo evitan fugas de información en el futuro hacia el entrenamiento.

- Registros limpios disponibles: 436.
- Registros útiles para modelos: 411.
- Registros de entrenamiento: aproximadamente 328.
- Registros de prueba: aproximadamente 83.

3.3. Modelos evaluados

Se evaluaron dos algoritmos de regresión:

- **KNN (K-Nearest Neighbors Regressor)**: predice a partir de los registros históricos más similares. Se utilizó escalado estándar en las variables de entrada, ya que KNN se basa en distancias y puede verse afectado si las variables tienen escalas muy diferentes.
- **SVR (Support Vector Regression)**: modelo de regresión, con kernel RBF. Se aplicó escalado estándar tanto a las variables de entrada como a la variable objetivo.

3.4. Resultados obtenidos

Modelo	RMSE (MW)	MAE (MW)	R^2
KNN	79.03	64.22	0.9147
SVR	192.49	169.76	0.4942

Tabla 2: Resultados comparativos de los modelos evaluados en la corrida final.

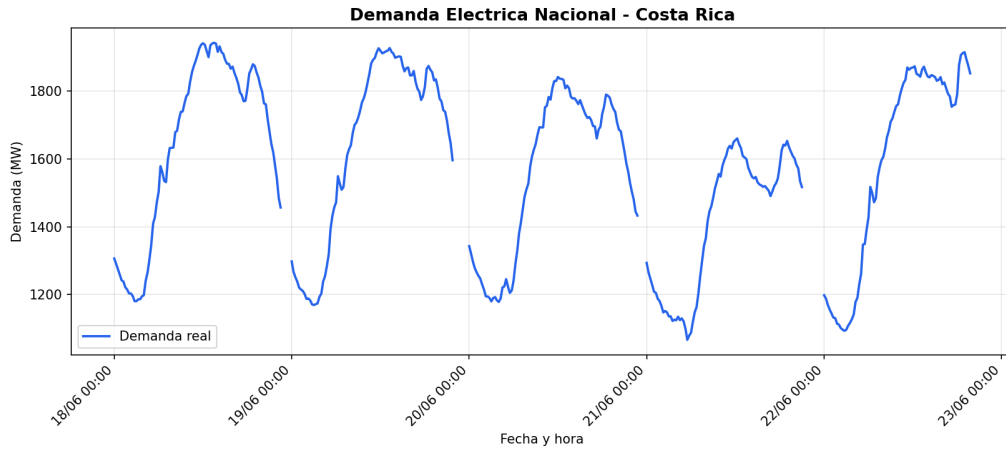


Figura 1: Curva de demanda real entre el 18 y el 22 de junio de 2026. Los cortes representan huecos temporales reales detectados en los datos.

3.5. Predicciones contra demanda real

La Figura 2 compara la demanda real contra las predicciones de KNN y SVR sobre el conjunto de prueba. Esta gráfica no muestra todo el periodo histórico, sino únicamente el tramo usado para evaluar los modelos, correspondiente al 20 % más reciente de los registros útiles. Visualmente, KNN sigue mejor la forma general de la demanda real, especialmente en los tramos de demanda alta cercanos al 22 de junio. SVR tiende a subestimar la demanda en una parte importante del conjunto de prueba. Esta observación coincide con las métricas cuantitativas: KNN obtuvo menor RMSE, menor MAE y mayor R^2 .

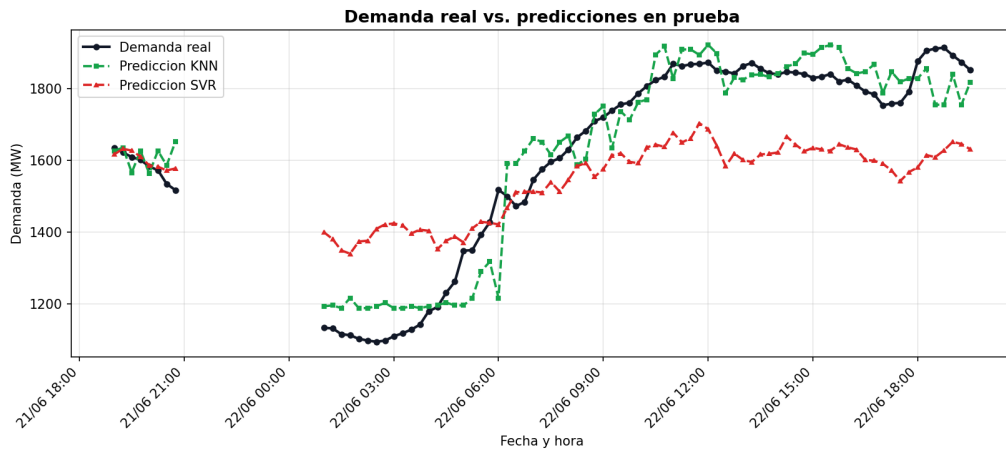


Figura 2: Demanda real vs. predicciones de KNN y SVR sobre el conjunto de prueba.

3.6. Comparación de métricas

La Figura 3 resume de forma gráfica la comparación de RMSE y MAE entre KNN y SVR. En ambos indicadores, KNN presenta errores menores que SVR, lo que confirma su selección como modelo recomendado en esta ejecución.

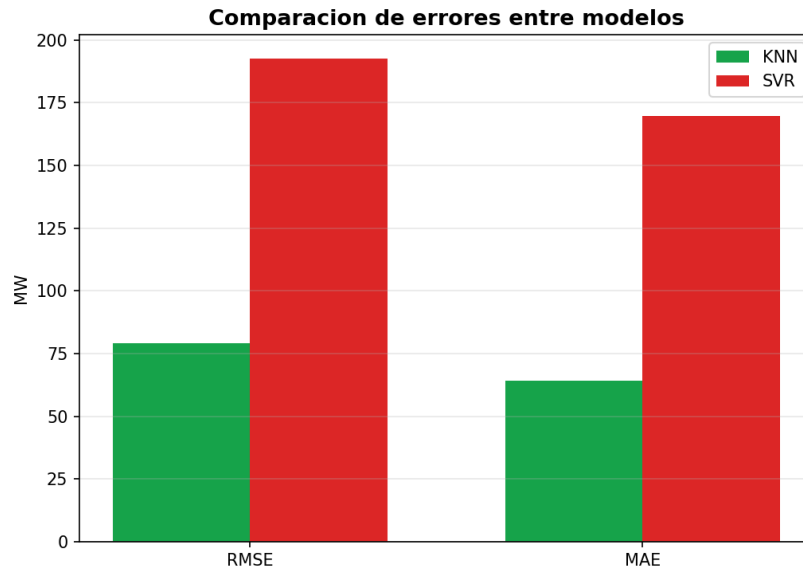


Figura 3: Comparación de RMSE y MAE entre KNN y SVR.

4. Diseño del Agente

4.1. Arquitectura

El agente implementado en `Agente.py` sigue un ciclo de razonamiento compuesto por cuatro etapas:

1. **Analizar intención:** interpreta la pregunta del usuario y determina si se solicita una predicción, una comparación de modelos o estadísticas del sistema.
2. **Seleccionar herramienta:** elige la función correspondiente, como predicción con KNN, predicción con SVR, comparación de modelos o estadísticas.
3. **Ejecutar herramienta:** invoca el modelo entrenado o la función estadística necesaria.
4. **Generar respuesta:** interpreta el resultado numérico y lo comunica, incluyendo tendencia esperada, margen de error y recomendación operacional.

4.2. Herramientas integradas

Herramienta	Descripción
Predicción KNN	Estima la demanda del siguiente intervalo de 15 minutos usando K-Nearest Neighbors.
Predicción SVR	Estima la demanda del siguiente intervalo usando Support Vector Regression.
Predicción con mejor modelo	Cuando el usuario no especifica modelo, el agente selecciona automáticamente el modelo con menor RMSE en la corrida actual.
Comparación de modelos	Ejecuta ambos modelos y recomienda el más preciso según las métricas de prueba.
Estadísticas del sistema	Retorna rango de fechas, intervalos faltantes, anomalías, demanda mínima, máxima, promedio y desempeño de modelos.
Visualización	Genera gráficas de demanda histórica, predicciones y comparación de métricas.

Tabla 3: Herramientas integradas al agente.

4.3. Ejemplo de interacción

Pregunta: ¿Cuál será la demanda en los próximos 15 minutos?

Intención detectada: mejor. El agente selecciona automáticamente KNN, por ser el modelo con menor RMSE en la corrida final.

Respuesta del agente:

Demanda actual: 1892.4 MW. Predicción a 15 minutos: 1817.0 MW. Cambio esperado: -75.5 MW, por lo que la demanda bajará. El error típico del modelo es aproximadamente ± 79.0 MW. El nivel de demanda se clasifica como alto, cercano al pico del sistema, por lo que se recomienda mantener reserva de generación disponible.

5. Reproducibilidad del Proyecto

La estructura del proyecto permite reproducir el flujo completo de procesamiento, entrenamiento, visualización y ejecución del agente. Los scripts principales puedan reproducirse siguiendo una secuencia de:

- `Preprocesamiento.py`: combina archivos, limpia datos, detecta huecos y genera `datos_limpios.csv`.
- `Modelos.py`: entrena KNN y SVR, calcula métricas y expone funciones de predicción.
- `Visualizacion.py`: genera las gráficas de demanda histórica, predicciones y métricas.
- `Agente.py`: implementa el agente y sus herramientas.

La estructura general del repositorio es la siguiente:

```
Proyecto2_IA_C23562/  
|-- OperacionTiempoReal.txt  
|-- OperacionTiempoReal2.txt  
|-- OperacionTiempoReal3.txt  
|-- OperacionTiempoReal4.txt  
|-- OperacionTiempoReal5.txt  
|-- Preprocesamiento.py  
|-- Modelos.py  
|-- Visualizacion.py  
|-- Agente.py  
|-- datos_limpios.csv  
|-- demanda_historica.png  
|-- predicciones_vs_real.png  
|-- comparacion_metricas.png  
|-- Proyecto2IA_C23562.pdf  
|-- README.md  
|-- MODELCARD.md
```

5.1. Repositorio

Repositorio de GitHub: <https://github.com/dguevara8/ucr-artificial-intelligence-II>

6. Limitaciones y Dificultades

- Aunque la corrida final contiene 436 registros limpios, el conjunto sigue siendo relativamente pequeño para capturar patrones semanales, efectos de feriados o variaciones estacionales.
- Se detectaron 28 intervalos faltantes en la serie temporal. Estos huecos se manejan explícitamente tanto en la visualización como en la construcción de variables de rezago, pero reflejan una limitación real de los datos disponibles.
- El mejor modelo depende de la cantidad y continuidad de los datos. En esta corrida KNN fue superior a SVR, pero esto no implica que siempre vaya a serlo. Por eso el agente compara las herramientas disponibles en cada ejecución.
- No se realizó una búsqueda exhaustiva de hiperparámetros.

- No se incluyeron variables externas como feriados, temperatura, tipo de día laboral o estacionalidad mensual.

7. Oportunidades de Desarrollo Futuro

- Ampliar el conjunto de datos con más días consecutivos descargados del CENCE, idealmente semanas o meses completos.
- Agregar otros modelos, como árboles de regresión o Random Forest.
- Automatizar la descarga periódica de datos desde la plataforma del CENCE.

8. Conclusiones

El proyecto permitió construir un agente inteligente capaz de predecir demanda eléctrica de corto plazo a partir de datos reales del CENCE. La implementación integra preprocesamiento, detección de anomalías, manejo de huecos temporales, entrenamiento de modelos, visualización y selección automática de la mejor herramienta de predicción.

En la corrida final, el modelo KNN obtuvo el mejor desempeño, con un RMSE de 79.03 MW, un MAE de 64.22 MW y un R^2 de 0.9147. Por esta razón, el agente lo seleccionó como modelo recomendado. Además, las gráficas generadas permiten interpretar visualmente tanto la serie histórica como el comportamiento de los modelos en el conjunto de prueba.

Los huecos observados en la gráfica histórica están justificados por la ausencia real de algunos intervalos de 15 minutos en los archivos descargados. El sistema los representa correctamente como cortes en la curva, evitando crear transiciones artificiales. Esto fortalece la validez del análisis, ya que el modelo y la visualización respetan la estructura temporal real de los datos.